

ML Ops Assignment 2

ML OPS assignment for Hugging Face Fine-Tuning, Experiment Tracking Model Deployment

Name: Balamurali Palaniyandi

Roll Number: G25AIT2070

1. Introduction & Objective

This project implements the pipeline by fine-tuning a pre-trained **DistilBERT** model on **UCSD Goodreads reviews** to classify books into **eight genres**. The work demonstrates end-to-end MLOps practices: production scripting, experiment tracking, cloud GPU training, evaluation with artifact logging, and model deployment.

Core ML Ops practices demonstrated:

- **Productionizing notebooks:** Modular Python scripts for data loading, training, evaluation, and deployment.
- **Experiment tracking:** Logging metrics, hyperparameters, and artifacts via **Weights & Biases (W&B)**.
- **Cloud GPU training:** Leveraging Kaggle's free GPU infrastructure for efficient model training.
- **Model deployment:** Publishing the fine-tuned model to the **Hugging Face Hub**.
- **Reproducibility:** Scripts and tracked experiments ensure the workflow is repeatable.

The focus of this project is on implementing a complete end-to-end MLOps workflow — covering data ingestion, model training, and deployment — using modern open-source tools, rather than solely optimizing raw accuracy.

2. Dataset

Attribute	Value
Source	UCSD Book Graph
Genres	poetry, children, comics_graphic, fantasy_paranormal, history_biography, mystery_thriller_crime, romance, young_adult
Features	Review text only
Size	8 genres $\times$ 2,000 reviews sampled = $\sim$ 16,000 total
Preprocessing	Tokenized with DistilBERT tokenizer; truncated/padded to 512 tokens
Split	800 training / 200 test per genre (80/20 split)

The dataset is loaded via HTTP streaming from UCSD mirrors, sampled to 2,000 reviews per genre for computational efficiency, and split stratified by genre.

3. Model & Training Setup

Model: distilbert-base-cased

Tokenizer: DistilBertTokenizerFast

Classes: 8 genres

Max length: 512 tokens

Device: CUDA (Kaggle T4 x2)

Hyperparameters:

Parameter	Value
Epochs	3
Batch size (train)	10
Batch size (eval)	16
Learning rate	5e-5
Warmup steps	100
Weight decay	0.01
Evaluation strategy	steps
Logging	every 100 steps

Why DistilBERT is suitable for this assignment:

- Efficiency: With only 66 million parameters (compared to BERT-Base's 110M), it trains significantly faster on a free Kaggle GPU (T4 x2), allowing us to complete 3 epochs within $\sim$ 20 minutes.

- Task alignment: As a bidirectional encoder, DistilBERT is well-suited for sequence classification tasks like genre prediction, where understanding context from both directions of the text is critical.
- Accessibility: Being hosted on the Hugging Face Hub, it integrates seamlessly with the transformers library and the Trainer API, enabling rapid prototyping without manual architecture implementation.
- Cased variant: We chose the cased (base-cased) version because book reviews often contain proper nouns (author names, book titles) where capitalization carries semantic value that could aid classification.

Key Differences

Cased vs Uncased:

- Cased models keep capitalization distinctions, useful for tasks like NER where “US” vs “us” matters.
- Uncased models simplify text by lowercasing, often improving robustness for noisy text (social media, reviews).

DistilBERT vs BERT:

- DistilBERT is smaller, faster, and lighter (40% fewer parameters, 60% faster) while keeping ~97% of BERT’s accuracy.
- BERT is larger and more resource-intensive, but slightly more accurate on complex tasks.

Practical Guidance

- Choose DistilBERT-uncased for efficiency and general tasks like sentiment analysis.
- Use DistilBERT-cased if capitalization is critical (NER, proper nouns).
- Opt for BERT-base-cased when accuracy is more important than speed.
- Select BERT-base-uncased for broad NLP tasks where case distinctions are less relevant.

In summary, DistilBERT offers the best trade-off between performance, computational cost, and ease of deployment for an educational MLOps assignment.

4. Implementation Highlights

Kaggle Secrets Management

API tokens are stored securely via **Kaggle Secrets** rather than hard-coding:

- WANDB_API_KEY — W&B authentication.
- HF_TOKEN — Hugging Face Hub write access.

```
from kaggle_secrets import UserSecretsClient
secrets = UserSecretsClient()
WANDB_API_KEY = secrets.get_secret('WANDB_API_KEY')
HF_TOKEN = secrets.get_secret('HF_TOKEN')
```

Weights & Biases Integration

W&B is initialized before training with the full hyperparameter configuration:

```
wandb.init(
    project='mlops-assignment2',
    name='distilbert-goodreads-run-1',
    config={'model': model_name, 'epochs': 3, ...}
)
```

TrainingArguments reports to wandb, and the Trainer streams loss, evaluation metrics, and model checkpoints automatically.

Evaluation Metrics

The custom compute_metrics function returns **accuracy** and **weighted F1** to the Trainer:

```
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    acc = accuracy_score(labels, preds)
    f1 = precision_recall_fscore_support(labels, preds, average='weighted')
    [2].mean()
    return {'accuracy': acc, 'f1': f1}
```

Hugging Face Hub Deployment

The fine-tuned model and tokenizer are pushed to the Hub under the repository balamuralipalani/distilbert-goodreads-genres:

```
from huggingface_hub import login

# HF_TOKEN was already loaded from Kaggle Secrets (see Task 4)
login(token=HF_TOKEN)
# Push model and tokenizer
model.push_to_hub("balamuralipalani/distilbert-goodreads-genres")
tokenizer.push_to_hub("balamuralipalani/distilbert-goodreads-genres")
# Record the link in W&B
wandb.init()
wandb.run.summary["huggingface_model"] = \
    "https://huggingface.co/balamuralipalani/distilbert-goodreads-genres"
wandb.finish()
```

Inference from Hub

A demonstration cell loads the deployed model directly from Hugging Face Hub and performs interactive genre prediction on user-supplied book reviews, confirming the deployment is functional.

5. Evaluation Results

After 3 epochs of fine-tuning, the model was evaluated on the held-out test set.

Metric	Score
Accuracy	0.570
F1 Score	0.586
Eval Loss	2.404

6. Submission Links

Resource	Public Link
GitHub Repository	https://github.com/balamuralipalani/ML_Ops
Kaggle Notebook	https://www.kaggle.com/code/balamuralipalaniyand/mlops-assignment-2-fine-tuning-classification
Hugging Face Model	https://huggingface.co/balamuralipalani/distilbert-goodreads-genres
W&B Dashboard	https://wandb.ai/g25ait2070-indian-institute-of-technology-jodhpur/mlops-assignment2

7. Challenges Faced & Learnings

- Kaggle notebook import error:** For some reason, uploading the notebooks contents failed.
 - Resolution:** Imported entire notebook by import option under code section rather than home -> notebooks.
- Accelerator Error:** Faced accelerator error while training the model. It's due to the token exhaustion for distilbert which supports maximum of 30552 tokens only
 - Resolution:** By mistake, I have copied the training model for 2 times in various places. commenting one of the trainer call resolved this problem.
- WandB Reporting issue:** First couple of times my execution results were not reported to WandB dashboard.
 - Resolution:** There was a command under training arguments was mentioned as `report_to=[]`, updated this to `report_to='wandb'` and started seeing my dashboard.

8. Conclusion

This project effectively demonstrates the implementation of a complete end-to-end MLOps pipeline for the Goodreads genre-classification task. The workflow integrates all critical stages: data preparation, model fine-tuning with live experiment tracking, evaluation with artifact logging, and deployment to a public model repository.

Each component is seamlessly connected through modern open-source tools, ensuring transparency and reproducibility. The tracked configurations in Weights & Biases and the versioned model hosted on the Hugging Face Hub provide a robust foundation for experiment management and model governance.

Overall, the project highlights the practical application of MLOps principles, emphasizing automation, scalability, and reproducibility over raw model accuracy, thereby aligning with best practices in contemporary machine learning operations.

Future Work

- Building upon the successful implementation of the end-to-end MLOps pipeline, several enhancements can be pursued to further strengthen automation and scalability.
- CI/CD Integration: Incorporate continuous integration and continuous deployment pipelines to automate model retraining and deployment workflows.
- Model Monitoring: Establish real-time monitoring for model performance and data drift using tools such as EvidentlyAI or MLflow.
- Automated Retraining: Schedule periodic retraining triggered by new data or performance thresholds to maintain model accuracy over time.
- Containerization & Orchestration: Deploy models using Docker and Kubernetes for scalable, production-grade environments.
- Enhanced Experiment Tracking: Extend W&B integration to include hyperparameter optimization and dataset versioning for deeper reproducibility insights.

These improvements will evolve the current workflow into a fully automated, production-ready MLOps ecosystem capable of continuous learning and robust model lifecycle management.
